UNIT 23 90 min

Working Like a Python Developer

Input from users, generators, match statements, scope keywords, identity versus equality, testing with assert and unittest, virtual environments with pip, and Jupyter or Colab.

By the end of this unit you will be able to

- Read user input with `input()` and convert it safely
- Write a generator with `yield` and explain why it saves memory
- Use a `match` statement to choose among patterns
- Explain `global` and `nonlocal`, and when to avoid them
- Tell `is` from `==`
- Write a test with `assert` and `unittest`
- Create a virtual environment and install packages with pip
- Run notebook code in Jupyter or Google Colab

Note

This is a reference unit. Read it before Units 19 to 21, which use Colab and pip. The sandbox cannot run `input()`, so those examples are shown as code to run on your computer or in Colab.

Reading input

`input(prompt)` pauses the program, shows the prompt, and returns what the user typed as a **string**. Convert it before using it as a number. Anything a user types can be wrong, so pair the conversion with an exception check from Unit 11.

ask_age.py

```
def ask_age():
    while True:
        text = input("Your age: ")
        try:
            age = int(text)
        except ValueError:
            print("Please type a whole number.")
            continue
        if age < 0:
            print("Age cannot be negative.")
            continue
        return age

print("You are", ask_age())
```

Generators: values one at a time

A function that uses `yield` is a **generator**. Each `yield` hands back one value and pauses. The next value is computed only when it is asked for. A list of a million numbers is stored in memory all at once; a generator can produce the same numbers without storing them.

generator.py

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

print(list(countdown(3)))

total = 0
for value in countdown(4):
    total += value
print(total)
```

Generator expressions from Unit 12 are generators too. Use them when you only need to loop once over a large sequence.

match: choosing by pattern

The `match` statement (Python 3.10 and later) compares a value against several patterns. It is clearer than a long `elif` chain when you are checking one value against fixed options. The `_` pattern is the default.

match.py

```
def describe(code):
    match code:
        case 200:
            return "OK"
        case 404:
            return "Not found"
        case 500 | 503:
            return "Server problem"
        case _:
            return "Unknown"

print(describe(404))
print(describe(503))
print(describe(302))
```

global and nonlocal

Unit 6 showed that assigning a name inside a function creates a local variable. `global name` tells Python to assign the module-level variable instead. `nonlocal name` does the same for a variable in an enclosing function.

global_nonlocal.py

```
count = 0

def bump():
    global count
    count += 1

def make_counter():
    total = 0
    def add():
        nonlocal total
        total += 1
        return total
    return add

bump()
bump()
print(count)

counter = make_counter()
counter()
print(counter())
```

Watch out

Use `global` sparingly. A function that changes a global is hard to test and hard to reason about. Passing values in and returning results (Unit 6) is almost always clearer.

is versus ==

`==` asks whether two values are equal. `is` asks whether they are the **same object** in memory. Use `is` for `None` and for the singletons `True` and `False`; use `==` for everything else.

identity.py

```
a = [1, 2]
b = [1, 2]
c = a
print(a == b, a is b)
print(a is c)
print(None is None)
```

Testing with assert

`assert condition, message` stops the program with an `AssertionError` when the condition is False. It is a quick check during development. Tests are written the same way, but kept in files and run automatically.

assert_demo.py

```
def add_tax(price, rate=0.07):
    return round(price * (1 + rate), 2)

assert add_tax(100) == 107.0, "100 with tax should be 107.0"
assert add_tax(0) == 0, "zero stays zero"
print("all checks passed")
```

Testing with unittest

The `unittest` module (in the standard library) organizes tests into classes. Each method whose name starts with `test_` is a test. Run the file with `python test_tax.py`, and you see how many tests passed.

test_tax.py

```
import unittest

def add_tax(price, rate=0.07):
    return round(price * (1 + rate), 2)

class TestAddTax(unittest.TestCase):
    def test_basic(self):
        self.assertEqual(add_tax(100), 107.0)

    def test_zero(self):
        self.assertEqual(add_tax(0), 0)

if __name__ == "__main__":
    unittest.main()
```

> **Tip**
> Write the test before you fix a bug. If the test fails, you have reproduced the bug. When it passes, the bug stays fixed even after future changes.

Virtual environments and pip

A **virtual environment** is a private folder of packages for one project, so different projects can use different versions. Create one once per project, activate it, then install with `pip`. These commands are typed in a terminal, not in Python.

terminal (not Python)

```
python -m venv .venv
# Windows:   .venv\Scripts\activate
# macOS/Linux: source .venv/bin/activate
pip install numpy pandas scikit-learn matplotlib
pip freeze > requirements.txt
```

`pip freeze` records the exact versions you installed in `requirements.txt`. Someone else can recreate the environment with `pip install -r requirements.txt`. Keeping this file is the

difference between code that runs once and code that runs again.

Jupyter and Google Colab

A **notebook** mixes code cells with text, and each cell runs in the same session. It is ideal for exploring data and for lessons like these. **Google Colab** runs notebooks in your browser with NumPy, pandas, scikit-learn, matplotlib, and PyTorch already installed (PyTorch is used in Units 20 and 21). To install something extra in Colab, run `!pip install package-name` in a cell.

- Run a cell with Shift+Enter. The variables persist between cells.
- If results look wrong, use Runtime → Restart session and run everything from the top. Hidden state from earlier cells is a common source of confusing results.
- Notebooks are for exploring. Move code you rely on into a `.py` file with functions and tests.

Debugging with print and the debugger

The fastest debugger is a `print()` that shows the value you are unsure about, with a label. Remove it when you are done. For longer problems, the built-in debugger `breakpoint()` pauses the program at that line and lets you inspect variables. Type `c` to continue and `q` to quit.

Predict the output THINK FIRST

Reason through the behavior, then check it.

```
def g():
    yield 1
    yield 2
print(list(g()))
```

Answer: `[1, 2]`

```
x = None
print(x is None)
```

Answer: `True`

```
print([1] == [1], [1] is [1])
```

Answer: `True False`
Two separate list objects with the same contents.

```
def f(n):
    match n:
        case 0:
            return "zero"
        case _:
            return "other"
print(f(5))
```

Answer: `other`

```
count = 1
def f():
    global count
    count = 5
f()
print(count)
```

Answer: `5`

Practice

▼ **Exercise 1 — Even numbers generator**

Write a generator `evens(limit)` that yields the even numbers from 0 up to but not including `limit`. Print `list(evens(10))`.

▼ **Exercise 2 — Test a function**

Write a function `is_leap(year)` using the rule: divisible by 4, except centuries that are not divisible by 400. Write two `assert` statements: 2024 is leap, and 1900 is not.

Discussion

1. Why does a generator matter for a file with ten million lines? What would a list of those lines cost?
2. You inherit a project with no `requirements.txt`. What would you do first, and why?
3. Which tool in this unit would have saved you the most time in a project you already finished?

Key terms

input()

A built-in that shows a prompt and returns the user's typed text as a string.

Generator

A function that uses `yield` to produce values one at a time, pausing between them.

yield

A statement that returns a value from a generator and pauses until the next value is requested.

match statement

A statement that compares a value against patterns and runs the first pattern that fits (Python 3.10+).

global / nonlocal

Keywords that let a function assign a variable from the module level (`global`) or an enclosing function (`nonlocal`).

is

The identity operator. True when two names refer to the same object in memory.

assert

A statement that raises `AssertionError` when its condition is False, used for quick checks and tests.

unittest

The standard-library testing framework. Tests are methods named `test_...` inside a `TestCase` class.

Virtual environment

An isolated folder of installed packages for one project, created with `python -m venv`.

pip

The package installer for Python. Installs libraries from the Python Package Index.

Notebook

A document of code and text cells, each runnable in a shared session. Jupyter and Google Colab both use notebooks.

See all terms in the glossary →